

Ciclo II

Desarrollo de Aplicaciones Web

Sesión 4: Arquitectura de Funciones

Mg. Dennis Antonio Pérez Escobar

Agenda de la Sesión

- Recapitulación: Sesión 3
- 2.4. Funciones en JavaScript
- Sintaxis y Llamada a funciones
- Paso de Argumentos y `return`
- Ámbito de Variables (Scope)
- Literales de Función y Arrow Functions
- Funciones como Objetos (First-class)
- Manejadores de Eventos (Event Handlers)
- Funciones Recursivas

Recapitulando: Sesión 3

Arreglos (Arrays)

Entendimos que un arreglo es una estructura ordenada de datos indexada desde el **0**. Su propiedad principal es `.length`.

Matrices "Asociativas" y Mutación

Aclaramos que en JS no usamos arreglos con texto, usamos **Objetos**. Además, vimos cómo alterar arreglos con `push()`, `pop()`, `shift()` y `unshift()`.

Matrices Multidimensionales

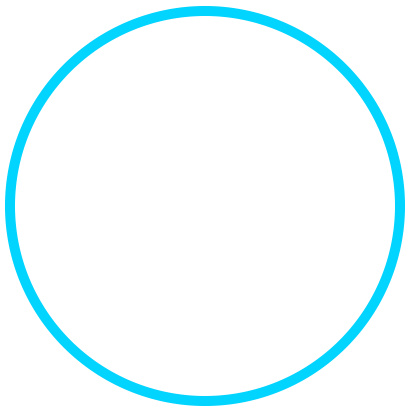
Arreglos dentro de arreglos. Se acceden mapeando coordenadas (X, Y) con corchetes dobles:

```
matriz[filas][columnas]
```

Métodos Modernos

Conocimos alternativas modernas a los bucles tradicionales: `forEach()`, `map()`, y `filter()` para recorrer y transformar listas.

Conociendo al Docente



Mg. Dennis Pérez

- Máster en Arquitectura de Software e Ing. en Ciencias de la Computación (UDB).
- CEO y Fundador de Tecnova Sv ("Soluciones Tecnológicas Empresariales").
- Especialista en desarrollo corporativo moderno (.NET, Angular, JS nativo).
- Diseñador de algoritmos eficientes y arquitecturas limpias orientadas a eventos.

2.4. ¿Qué es una Función?

Una función es el bloque de construcción fundamental de cualquier aplicación escalable. Es un subprograma diseñado para realizar una tarea específica.

La Analogía de la Máquina:

- **Materia Prima:** Los datos que le enviamos (Argumentos).
- **Mecanismo interno:** La lógica encapsulada.
- **Producto final:** El valor procesado que nos devuelve (Retorno).

Principio Arquitectónico

**"Una función debe hacer
UNA SOLA COSA
y debe hacerla bien."**

(Single Responsibility Principle - SOLID)

2.4. Sintaxis y Llamada (Invocación)

1. Declaración (Crear la Máquina)

Se usa la palabra reservada `function`, seguida del nombre y paréntesis. El bloque de código no se ejecuta automáticamente, solo se guarda en memoria.

```
function saludarBase() {
  console.log("¡Hola, estudiante!");
}
```

2. Invocación (Encender la Máquina)

Para ejecutarla, escribimos su nombre seguido de paréntesis `()`. Sin los paréntesis, solo estaríamos referenciando la función, no ejecutándola.

```
// En el código JS
saludarBase();

// Resultado en consola:
// "¡Hola, estudiante!"
```

2.4. Argumentos y Devolución de Valores

Una función adquiere poder real cuando puede recibir datos dinámicos, procesarlos y devolver un resultado útil para el resto del sistema.

Parámetros / Argumentos

Son variables locales que reciben la "materia prima". Se definen entre los paréntesis.

```
function sumar(a, b) {
  let total = a + b;
}
```

Devolución (return)

Termina la ejecución de la función y "escupe" el resultado hacia afuera. Si no hay return, devuelve undefined .

```
function sumar(a, b) {
  return a + b;
}
```

```
let facturaTotal = sumar(150, 45); // Devuelve 195 y lo guarda
```

2.4. Ámbito de Variables (Scope)

El "Scope" define la vida y la visibilidad de una variable. JavaScript usa un ámbito léxico (basado en bloques `{}`).

La Regla de Las Vegas

"Lo que se declara dentro de una función, se queda (y muere) dentro de esa función."

```
let usuarioGlobal = "Dennis"; // Scope Global

function procesarSeguridad() {
  let tokenLocal = "XYZ-123"; // Scope Local
  console.log(usuarioGlobal); // Funciona
}

procesarSeguridad();

// Error crítico: tokenLocal no existe aquí
console.log(tokenLocal); 🚨
```


2.4. Funciones como Objetos (First-Class)

En JavaScript, las funciones son **Ciudadanos de Primera Clase**. Esto significa que internamente son objetos y pueden ser tratadas como cualquier otra variable.

- Pueden ser asignadas a una variable.
- Pueden guardarse dentro de un Arreglo.
- **Poder máximo:** Pueden enviarse como argumentos a otra función (conocidas como *Callbacks*).

Ejemplo de Asignación y Callback

```
// Asignada a variable
const motorCalculo = function(x) {
  return x * 2;
};

// Enviada como argumento a un método
let datos = [1, 2, 3];
datos.map(motorCalculo);
// Resultado: [2, 4, 6]
```

2.4. Literales y Arrow Functions (ES6)

La evolución de JS nos trajo una sintaxis más limpia y profesional llamada **Funciones Flecha**. Eliminan la palabra `function` y manejan el `return` de forma implícita.

Función Literal (Anónima) Clásica

```
const sumar = function(a, b) {
  return a + b;
};
```

Arrow Function Estándar

```
const sumar = (a, b) => {
  return a + b;
};
```

Arrow Function Implícita (Nivel Senior)

Si la función solo tiene una línea, se quitan las llaves y el return es automático.

```
const sumar = (a, b) => a + b;
```

2.4. Manejadores de Eventos (El Gatillo)

Las funciones no siempre se llaman automáticamente. A menudo esperan a que el usuario interactúe con el DOM (clics, teclado, scroll).

Conectamos las funciones a la interfaz a través de **Event Handlers**.

- **HTML en línea (Evitar):** `onclick="procesar()"`
- **Listener Dinámico (Estándar):** JS "escucha" el botón de forma invisible.

```
// Forma profesional de inyectar eventos
const boton = document.getElementById("btnGuardar");

// La función actúa como Callback del evento
boton.addEventListener("click", () => {
    validarFormulario();
    guardarEnBaseDeDatos();
});
```

2.4. Funciones Recursivas

Una función recursiva es aquella que **se llama a sí misma** dentro de su propio bloque de ejecución.

Son complejas de leer, pero increíblemente potentes para recorrer estructuras profundas como árboles genealógicos, directorios de archivos o nodos anidados del DOM.

⚠ Obligatorio: Siempre deben tener una condición de corte para evitar colapsar la memoria (Stack Overflow).

Ejemplo Clásico: Cuenta Regresiva

```
function cuentaAtras(n) {
  // Condición de corte
  if (n === 0) {
    console.log("¡Despegue!");
    return;
  }

  console.log(n);
  // Se llama a sí misma
  cuentaAtras(n - 1);
}

cuentaAtras(3); // Imprime: 3, 2, 1, ¡Despegue!
```

Campos de Aplicación en la Industria

Todo sistema moderno interactivo es un ecosistema orquestado por funciones que se comunican entre sí.

Peticiones Asíncronas (Fetch/AJAX)

Funciones que se ejecutan en segundo plano solicitando datos a servidores sin bloquear la pantalla del usuario.

Validaciones de Seguridad

Módulos de funciones dedicadas puramente a limpiar, verificar e higienizar datos de formularios antes del envío.

Componentización (React/Angular)

En el desarrollo moderno, un "Componente Visual" (como un menú de navegación) es, bajo la superficie, simplemente una Función.

Funciones de Callback

Funciones que se pasan como parámetros para manejar respuestas de bases de datos o animaciones temporizadas.

¿Preguntas?

Siguiente Paso: Práctica Arquitectónica

Prepara tu entorno para estructurar módulos lógicos con Arrow Functions y manipulación del DOM.

